Cognitive Tours: Making Understanding a Team Deliverable in AI-Assisted Development

Margaret-Anne Storey
Department of Computer Science
University of Victoria, Canada
mstorey@uvic.ca

Abstract*—Generative AI and agentic development are accelerating software development, leading to concerns about technical debt. But two less visible forms of debt, cognitive debt and intent debt, are emerging as key risks to software health. *Cognitive debt* is the erosion of a team's shared understanding where no one can confidently explain how a system works or predict the impact of a change. *Intent debt* is the absence of clear goals, constraints, and rationale that explain what the system is for and guide how it should evolve. This paper presents *Cognitive Tours*, a tool designed to mitigate and repair cognitive and intent debt. Cognitive Tours is a web-based prototype for creating and following curated paths connecting user needs, design decisions, code, tests, and open questions. AI-assisted skills help audit existing waypoints and tours for possible problems or staleness and suggest candidates for new ones; humans decide what matters, verify relationships, and construct the narrative. This division of work deliberately preserves *productive friction*: the sensemaking involved in selecting, explaining, and revising a tour, that is intended to foster understanding. Our demonstration uses a tour of *Cognitive Tours itself* to introduce the tool, illustrate its design decisions, interaction mechanisms and an evaluation agenda. *Demo video: https://youtu.be/I2S5Ee3bzi8

***Index Terms*—program comprehension, AI-assisted development, cognitive debt, software intent, code tours.**

I. Introduction

Generative AI and agentic development are accelerating software development, allowing individuals and teams to generate and modify code faster than ever before. But many developers and teams are sharing how they struggle to keep track of the work they are doing and answer questions about the reason for its design, the constraints it must satisfy and the ability for the system to evolve to adapt to future requirements and changes [1]. Cognitive debt is held by a team, not only an individual: when the reasoning behind a change lives in one developer's head or one chat transcript, everyone else on the team, and that developer months later, inherits the gap.

Although developers may have access to the code, agent interaction histories and agents to explain how their systems work, developers struggle to maintain understanding and control of their system. Writing and modifying code has traditionally provided opportunities for developers to develop their understanding through implementation decisions and problem solving. Delegating these activities to AI may remove some of those opportunities. Furthermore, the *why* behind many of the decisions developers make is often not found in the source code (e.g., discarded ideas) and both humans and agents struggle to understand invisible but critical design decisions.

The *triple debt* model of software health draws attention to two concerns beyond technical debt: *cognitive debt*, the erosion of the shared understanding needed to work with a system, and *intent debt*, the loss or absence of explicit goals, constraints, and rationale [1]. A program may be technically sound and of high quality, but be difficult for its maintainers to explain or know how it can be safely evolved. Furthermore, a working and plausible implementation may not satisfy a requirement that was never adequately articulated or externalized.

Cognitive Tours provides a mechanism for developers relying on AI and/or agents to generate implementation to hand craft curated journeys through the reasons, decisions, implementation, evidence, and uncertainties that help tell the story behind a feature or task. The intended user of these tours is not only a newcomer onboarding to unfamiliar code, but also to the developer themselves who must later make changes to the system and recapture forgotten decisions and assumptions.

Tours are human-authored by default, with optional AI drafting that requires human verification. The author must decide what matters to record and which ones to omit so they do not occlude the important information that captures how software and design artifacts map to intent. The broad adoption and power of AI not only provides the urgent motivation for this tool, but it also supports the identification of new candidate and outdated waypoints and tours as the system evolves. The approach is designed to reduce the toil in creating and updating them, but not fully automate and replace understanding.

This paper demonstrates a web-based prototype through a *self-tour*: we use a tour of Cognitive Tours to explain the motivation and design of Cognitive Tours. This choice avoids an artificial sample application and provides a concrete, self-documenting narrative about the tool. Our contribution is a *tool and interaction design proposition*, not a validated claim (yet) that tours reduce cognitive or intent debt. However, the tool has been used by the author during its development and provided cognitive support across sessions and exposed a discrepancy between an ADR and recent implementation decision.

II. Background and Tool Positioning

20 years ago, the author introduced the *waypoint metaphor* for collaborative understanding and software navigation [2]. The *Tours* Eclipse plugin combined slides and live IDE navigation for programmer-to-programmer presentations and tours [3]. We revisit these design principles and adapt *Software Tours*

and *Waypoints* to today's era of AI-assisted development. This earlier attempt saw barriers to adoption due to challenges maintaining waypoints and tours as the software changed over time. Today, this toil can be mitigated with the use of AI and agent skills to help developers keep waypoints and tours up to date but preserve how creating them helps enrich understanding. The use of AI is limited to scaffolding, the developer is responsible for understanding.

Recent work has explored the potential value of guided code exploration. Taylor and Clarke studied Microsoft CodeTour, manually creating two annotated CodeTours, studying its potential for orientation and navigation [4]. LACY (FSE 2026) combines AI-generated content with expert-curated code tours, comprehension questions, and other onboarding support [5]. Its evaluation shows promising evidence for expert-guided onboarding. CodeMap (ICPC 2026) similarly explores human–AI collaboration and uses AI to dynamically generate hierarchical visualizations for exploring unfamiliar codebases [6].

Cognitive Tours differs from these tools in its application beyond code artifacts to design and intent artifacts, and its allocation of cognitive work. In addition to helping newcomers navigate code, it supports human-authored, revisable accounts connecting software intent, design decisions, implementation, tests, and unresolved questions. AI skills audit existing waypoints and tours and suggest additions, while humans select, interpret, connect, and explain what matters. We preserve this *productive manual friction* to foster developer understanding.

Cognitive Tours can be built before, during or at the end of a development session.

Bram[1] preserves agent-work sessions, human approvals, and associated development history. Cognitive Tours complements this approach by helping developers construct and maintain an explicit account of software intent and understanding. Romeo et al. [7] investigate drift between software design, implementation, and documentation, using links between UML diagrams and source code to identify inconsistencies as systems evolve. Cognitive Tours takes a complementary approach by connecting user needs, design rationale, code, and tests through curated navigation paths, with AI-assisted auditing to identify potentially stale or inconsistent explanations.

Balfroid et al. [8] explored LLM-generated code tours based on failing-test traces, finding that automated generation is feasible but can produce redundant and low-level explanations. Cognitive Tours instead uses AI to support discovery and maintenance while keeping developers responsible for the explanations and rationale.

III. Cognitive Tours

A. Main Features: Waypoints, Relationships, and Tours

Cognitive Tours relies on three core concepts: a *waypoint*, a *relationship* or link between two waypoints, and a *tour* of waypoints. A waypoint is a selected anchor or landmark that the developer considers important for understanding: a system goal, user need, feature, architecture or design decision, AI instruction, code component, test, risk, or unresolved question. Waypoints have both a description and optional commentary, and may reference existing project artifacts with links to (for example) a GitHub repository or file in an IDE (code, specs, ADRs etc). Named (but intentionally untyped to be flexible) relationships explain why nodes are connected, for example a feature *addresses* a user need, or a feature is *implemented by* a component, or *poses* an architectural question or decision. Waypoints are intentionally placed on the graph by the user to help them "preserve a cognitive map" that persists across time. A user composes a tour by selecting and ordering some of these waypoints to tell a coherent story for a particular feature, decision, or purpose. A tour normally traverses only some of the waypoint nodes in the underlying graph.

Tours are shared artifacts: currently a map is reached through a single URL with no setup, each tour records its author and when it was last changed, and each team works in its own map, so a tour one member writes becomes a path the rest of the team can follow and question.

Docent annotations provide human-curated commentary between waypoints, explaining what matters, what to notice, and how waypoints connect. Creating these annotations encourages developers to articulate rationale, including decisions and alternatives that may not be recoverable from the source code. This deliberate authoring effort is intended to support team understanding as a deliverable of AI-assisted development.

The browser interface is organized around a navigable waypoint graphical map and an explanatory, navigable tour view. The latter places and highlights the current stop in context and presents human-curated commentary (docent annotations). The tours may be presented and navigated in the graphical view or as a markdown file (which can be exported).

The tour authoring workflow allows the externalized information to evolve as the software or the team's understanding changes. Over time some waypoints and tours may become less relevant. Skills can be used to recommend waypoints and relationships. A waypoint is not assumed to be correct merely because it appears in a graph: an unresolved question or a missing evidence link is a legitimate outcome of a tour. This is particularly important when AI assistance can quickly produce plausible but unverified rationales.

B. AI-Supported Auditing, Human-Led Sensemaking

Cognitive Tours uses AI-assisted skills to help maintain the waypoint graph and its curated routes. These skills audit individual waypoints and entire tours, flagging material that may need revisiting, and suggesting new waypoints or tours. They are aids to inspection and upkeep, not agentic authors of the system's account of itself. A person still decides which suggestions are meaningful, verifies the underlying evidence, writes or revises the rationale and commentary, and determines the route through which someone else should encounter it. We deliberately retain the manual authoring work, but use AI to reduce tedious discovery and maintenance work (observed blockers to our implementation from 20 years ago).

[1]https://github.com/judell/bram

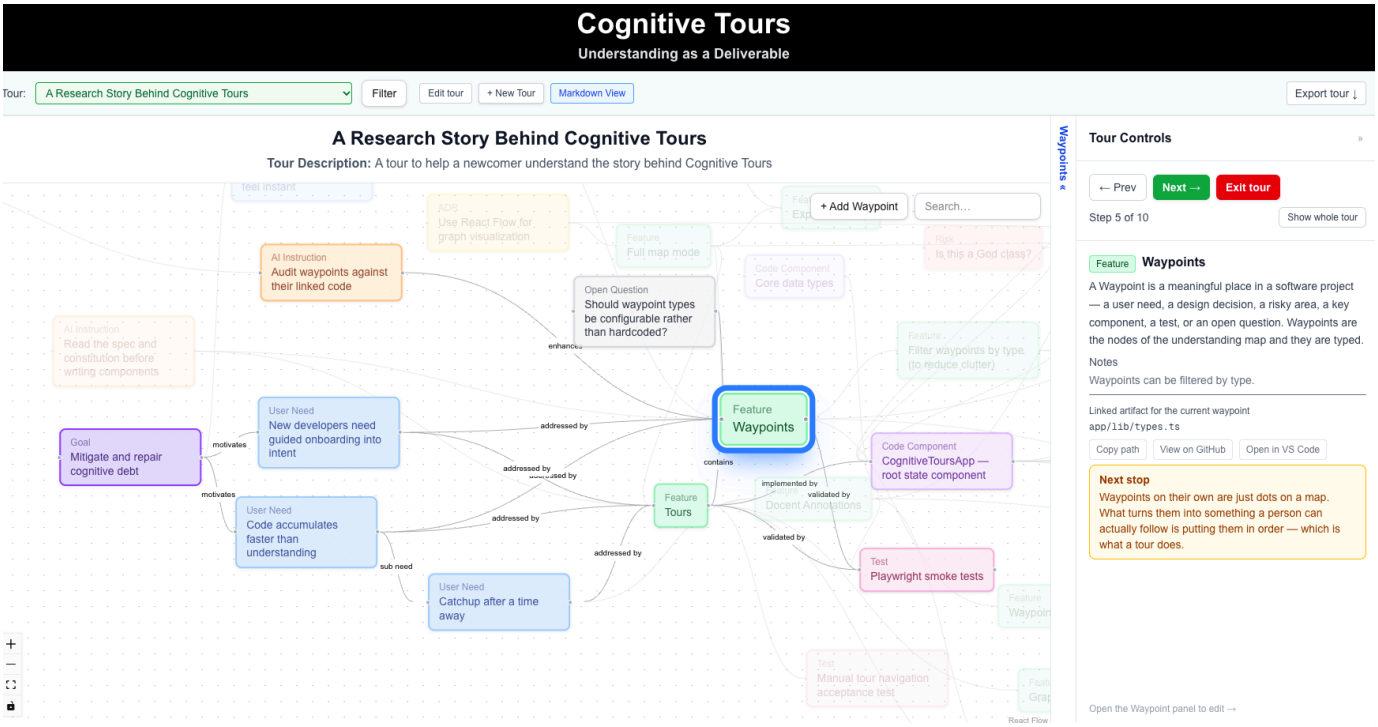


Fig. 1. The Cognitive Tours interface showing a guided tour of the application itself. The waypoint graph connects user needs, design decisions, and implementation artifacts, while the commentary panel guides developers through the reasoning behind the system.

Four Claude Code skills accompany the tool: 1) **audit-waypoints**: finds waypoint descriptions that no longer match their files; 2) **audit-tours**: finds unrecorded recent work and stale tour narratives; 3) **suggest-waypoint**: ranks recent work worth recording, then asks the author to explain it; and 4) **suggest-tour**: proposes where new waypoints fit, and asks the author to write each bridge between them.

All four skills run in Claude Code inside the project's repository (see the demo). They read the live map through the tool's `/api/data` endpoint, the git history since a given date, and the files that waypoints link to. They never write to the map. The two audit skills report findings one at a time, such as a waypoint description that no longer matches its linked file, a tour stop narrative that names a component that has since moved, or a recently changed file that appears in no tour; the two suggest skills rank recent work worth recording and supply only the fact fields (label, type, file path, edges, stop order), followed by questions for the author. The author decides which findings to act on, writes descriptions and narratives (a skill drafts content only on request, after first prompting the author to try).

IV. Demonstration: A Tour of Cognitive Tours

We demonstrate Cognitive Tours through a tour of the application itself, rather than an artificial example. A publicly accessible demonstration is available at https://cognitive-tours-demo.vercel.app/. To explore the tool, visitors can select the "A Research Story Behind Cognitive Tours" tour from the application's tour selector and navigate through its waypoints using the Next control. The self-referential example (shown in the video) serves two purposes: it introduces the design and motivation of Cognitive Tours while illustrating how a curated tour can communicate the story behind a software system (see Fig. 1). This particular tour was created to onboard a new team member.

Understanding why the system exists. The tour begins with the motivating problem: AI-assisted development can accelerate software production faster than developers can build and share an understanding of what has been created. The initial waypoints establish the user needs and design goals.

Following the design rationale. The tour then connects these goals to features and design decisions. Rather than navigating an unstructured collection of source files, users follow an ordered sequence of waypoints accompanied by contextual commentary (docent annotations). The underlying graph makes relationships between goals, features, decisions, and components visible.

Connecting intent and implementation. The tour has waypoints representing intended user goals, design decisions, implementation components and tests. The connections illustrate how a tour supports movement between explanations of why a feature exists and the technical artifacts that realize it.

Identifying open questions or gaps in understanding Finally, the tour presents an unresolved question about the system, including if waypoint types should be configurable. Such questions are part of the representation rather than problems to hide: they indicate where further investigation, evidence, or human judgment is needed.

```
> /suggest-waypoint since 2026-10-20
Read the map: 59 waypoints, 73 edges,
5 tours; 28 commits since Oct 20.

Worth recording, in order:
1. ADR: One site and database per team
   <- contradicts ADR 007 (still Accepted)
2. Feature: Team sites
3. Feature: File links that work for everyone
4. Feature: Backups for every site
5. Risk: Team sites are open

> 1
- What was decided, in your words?
- Why was it needed? What would confuse
  someone without it?
- What else did you consider, and why not?

> [AUTHOR'S ANSWER WOULD APPEAR HERE]
```

Fig. 2. `/suggest-waypoint` ranks unrecorded work and asks the author about the top decision; the author writes the answer, and the waypoint.

The demonstration illustrates Cognitive Tours as both a navigational artifact and a medium for communicating and revising understanding. Importantly, the proposed value lies not only in following a tour but also in the human work of selecting, connecting, explaining, and maintaining its waypoints.

V. Early Experience and Future Work

Poor understanding is not only a risk for newcomers. Cognitive debt also accrues when we move fast. In one evening of AI-assisted work preparing the tool for a fall course, the tool's author made more than 20 commits: a separate site and database for each of the student teams, project labels, links to each team's GitHub repository, and nightly backups. Although the author used Cognitive Tours to mitigate cognitive and intent debt through the tool development journey, during this particular coding session none of this was recorded by the author in the project's own Cognitive Tours map. The next day, a skill was run by the author to compare recent commits against the map to surface what may be missing and should be recorded for future sessions. To the author's surprise, it ranked first a design decision that contradicted an architectural decision record that was outdated. Had a collaborator joined the project the next day, the map would have told them the opposite of what the code did. The skill encouraged the author to walk through what was built and update and add new waypoints and tours to capture this work. Figure 2 shows an excerpt from this interaction. The skill reads the live map and the commits since a given date, ranks what is worth recording (decisions first, flagging any that contradict an existing ADR), and then asks questions rather than writing the waypoint. It supplies only the fact fields (label, type, file, edges); the description comes from the author's own answers (unless overridden).

Together, the self-tour and the AI-assisted auditing example demonstrate both sides of Cognitive Tours: following an existing account to understand a system, and critically maintaining that account as the system evolves.

We plan to compare typical AI-generated code explanations, fully AI-generated tours, and human authored Cognitive Tours supported by AI auditing. We will assess orientation, behavioral prediction, explanation quality, and knowledge transfer, while considering the additional effort required to create and maintain tours. We will also assess how cognitive tours can be used to foster understanding across a team, exploring how team members both curate and navigate each other's tours.

VI. Discussion and Limitations

The main premise of Cognitive Tours is not simply to provide another explanation of code. Instead, developers deliberately select what matters, articulate why, reconcile intent with implementation evidence, and preserve these interpretations as software evolves. AI-assisted skills surface candidates and discrepancies, *but* the human author constructs and revises the account. We deliberately retain this *productive friction*, hypothesizing that it supports the development of human understanding rather than merely the production of documentation. This approach introduces costs and risks. Tours and links may become stale, AI suggestions may be inaccurate, and curated paths may omit important steps. Human curation can become an unnecessary burden. Whether the focus on manual authorship leads to more accurate, durable, and shared understanding than AI-generated explanations remains future work.

Acknowledgement of AI Assistance. ChatGPT was used to suggest wording, improve clarity, and assist with manuscript revisions and formatting. The author critically reviewed and refined all text and takes full responsibility for the manuscript's content and claims. Claude Code was used to assist with implementation of the prototype under the author's direction.

References

[1] M.-A. Storey, "From technical debt to cognitive and intent debt: Rethinking software health in the age of AI," *ACM Queue*, vol. 2, no. 2, Jun. 2026. [Online]. Available: https://doi.org/10.1145/3807966

[2] M.-A. Storey, L.-T. Cheng, R. I. Bull, and P. C. Rigby, "Shared waypoints and social tagging to support collaboration in software development," in *Proceedings of the ACM Conference on Computer Supported Cooperative Work (CSCW)*, 2006, pp. 195–198.

[3] L.-T. Cheng, M. Desmond, and M.-A. Storey, "Presentations by programmers for programmers," in *Proceedings of the International Conference on Software Engineering (ICSE)*, 2007, pp. 788–792.

[4] G. Taylor and S. Clarke, "A tour through code: Helping developers become familiar with unfamiliar code," in *Proceedings of the Psychology of Programming Interest Group (PPIG)*, 2022, pp. 114–126. [Online]. Available: https://ppig.org/files/2022-PPIG-33rd-taylor.pdf

[5] Z. B. Kara, A. İsmail, E. Ateş, İ. N. Tamcı, Z. İyigün, S. Ş. Aslangül, Ö. Devran, B. M. Uçar, and E. Tüzün, "LACY: Simulating expert mentoring for software onboarding with code tours," in *Companion Proceedings of the ACM Symposium on the Foundations of Software Engineering (FSE)*, 2026, pp. 450–460.

[6] J. Gao, Y. Xue, X. Xie, J. Cao, S. Thant, E. Lee, and B. Xu, "Understanding codebase like a professional! human–AI collaboration for code comprehension," in *Proceedings of the IEEE/ACM International Conference on Program Comprehension (ICPC)*, 2026, pp. 343–354.

[7] J. Romeo, M. Raglianti, C. Nagy, and M. Lanza, "Capturing and understanding the drift between design, implementation, and documentation," in *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension (ICPC)*. ACM, 2024, pp. 382–386.

[8] M. Balfroid, B. Vanderose, and X. Devroey, "Towards llm-generated code tours for onboarding," in *Proceedings of the Third ACM/IEEE International Workshop on NL-Based Software Engineering (NLBSE)*. ACM, 2024, pp. 65–68.